

Analyse approfondie du fichier test_suite.py : L'Assurance Qualité (QA)

Ce fichier n'est pas exécuté par les utilisateurs de votre site. Il est exécuté par **vous** (le développeur) ou par des robots (CI/CD - Intégration Continue) avant chaque mise à jour. Il utilise la bibliothèque standard unittest, le cadre de test historique de Python. L'objectif est double :

1. **Tests Unitaires (Unit Tests)** : Vérifier que chaque petit boulon (l'algorithme) fonctionne isolément.
2. **Tests d'Intégration** : Vérifier que l'assemblage (Flask + Algo + OpenWeather) fonctionne ensemble.

1. Imports et Configuration de l'Environnement

Le début du fichier prépare le terrain. C'est la mise en place du "laboratoire".

```
import unittest
from unittest.mock import patch, MagicMock
import sys
import os
# ...
sys.path.append(os.path.dirname(...)) # Hack pour importer les modules parents
from app import app
from algo import PortfolioBrain
```

Analyse "Ingénierie" :

- **unittest vs pytest** : Bien que pytest soit très populaire aujourd'hui, utiliser unittest (inclus dans Python) montre que vous savez travailler sans dépendances externes lourdes. C'est "Pythonic".
- **unittest.mock (Le concept clé)** :
 - Dans un test, on ne veut **jamais** dépendre d'Internet.
 - Imaginez que vous lancez vos tests dans un train sans Wifi. Ils doivent réussir.
 - Imaginez que l'API OpenWeather est en panne. Vos tests ne doivent pas échouer pour autant (ce n'est pas votre code qui est cassé).
 - **La solution** : Le **Mocking**. On remplace requests.get (qui va sur Internet) par un "imitateur" (MagicMock) que l'on contrôle totalement. C'est comme utiliser une

doublure pour une cascade dangereuse au cinéma.

- **Le Hack sys.path** : Comme vos tests sont souvent dans un sous-dossier ou à côté, Python ne trouve pas toujours app.py. Cette ligne manipule le chemin de recherche pour forcer Python à trouver vos modules.

2. Tests Unitaires du Cerveau (TestPortfolioBrain)

Ici, on teste la mathématique pure de algo.py. Pas de réseau, pas de Flask, juste des chiffres.

Vérification de l'État Initial

```
def test_initial_values(self):
    self.assertEqual(self.brain.srtd, 3.0)
    # Calcul : 3.0 + 4*0.5 = 5.0
    self.assertEqual(self.brain.get_timeout(), 5.0)
```

- **Objectif** : S'assurer que les constantes définies dans l'algo sont bien respectées au démarrage. C'est un "Sanity Check" (Vérification de santé mentale).

Simulation du "Happy Path" (Algorithme de Jacobson)

```
def test_jacobson_update_success(self):
    # On insiste 50 fois...
    for _ in range(50):
        self.brain.update(observed_latency=0.1, success=True)

    self.assertLess(self.brain.get_timeout(), 5.0)
```

- **Le Concept de Convergence** : L'algorithme de Jacobson est une moyenne mobile. Il ne change pas instantanément (pour éviter l'instabilité).
- **Pourquoi une boucle ?** Pour prouver qu'après une série de bonnes connexions (0.1s), le timeout (qui était à 5s) descend progressivement pour s'adapter à la rapidité du réseau. C'est la preuve mathématique que votre algo apprend.

Simulation de l'Échec (Algorithme de Karn)

```
def test_karn_penalty_failure(self):
```

```
# ...
self.brain.update(observed_latency=10.0, success=False)
# Le timeout doit doubler
expected = min(10.0, initial_timeout * 2)
self.assertEqual(self.brain.get_timeout(), expected)
```

- **Vérification de la Punition** : On teste la logique binaire : "Si ça plante, tu doubles". On vérifie aussi le bornage ($\min(10.0, \dots)$) pour être sûr qu'on ne dépasse jamais le plafond UX défini.

3. Tests d'Intégration de l'API (TestGeoMeteoApp)

Ici, on lance une "fausse instance" de Flask (self.client) et on lui envoie des requêtes HTTP comme le ferait un navigateur.

Setup et Teardown

```
def setUp(self):
    self.client = app.test_client() # Le faux navigateur
```

- Pour chaque test, on recrée une application neuve. Cela garantit que le test A n'influence pas le test B (Isolation des tests).

Le Mocking en Action (@patch)

C'est la partie la plus technique du fichier.

```
@patch('requests.get') # On intercepte TOUS les appels requests.get
def test_autocomplete_mock(self, mock_get):
    # 1. On programme la doublure
    mock_resp = MagicMock()
    mock_resp.status_code = 200
    mock_resp.json.return_value = [{"name": "Paris", ...}]
    mock_get.return_value = mock_resp

    # 2. On exécute le code
    response = self.client.get("/autocomplete?q=Paris")

    # 3. On vérifie
```

```
self.assertEqual(data[0]['label'], "Paris, FR")
```

- **L'Injection de Dépendance virtuelle :**
 - Le décorateur `@patch` dit à Python : *"Pendant l'exécution de cette fonction, si quelqu'un appelle `requests.get`, ne le fais pas vraiment. Utilise `mock_get` à la place."*
 - On configure ensuite `mock_get` pour qu'il renvoie exactement ce qu'on veut. Ici, on simule qu'OpenWeather a répondu parfaitement.
- **Intérêt :** On teste que **notre** code (`app.py`) sait lire et transformer la réponse d'OpenWeather, sans avoir besoin d'Internet.

4. Tests de Résilience (Unhappy Paths)

Un bon ingénieur ne teste pas seulement quand tout va bien. Il teste surtout quand tout va mal.

Simulation des Pannes Externes

```
@patch('requests.Session.get')
def test_api_connection_error(self, mock_get):
    # On simule un CRASH réseau violent (Exception Python)
    mock_get.side_effect = requests.exceptions.ConnectionError("Pas d'internet")

    response = self.client.get("/?city=Paris")
    self.assertEqual(response.status_code, 200)
```

- **side_effect :** Au lieu de renvoyer une valeur, le Mock va lever une erreur (Crash).
- **Vérification de la Robustesse :** On vérifie que notre serveur Flask n'explose pas (Erreur 500 Internal Server Error) mais qu'il attrape l'exception proprement et renvoie une page valide (200 OK) avec un message d'erreur utilisateur.

Test de Sécurité (Injection)

```
def test_input_sanitization_injection(self):
    response = self.client.get("/?city=<script>alert('hack')</script>")
    self.assertNotIn(b"<script>alert", response.data)
```

- Vérifie que le validateur dans `app.py` fait son travail et rejette ou nettoie les tentatives de piratage XSS.

5. Tests "Senior" : Logique Métier et Cache

Vérification de la Logique Complète

```
def test_full_weather_api_logic(self, mock_get):  
    # ... Mock complet ...  
    self.assertEqual(data['temperature'], 25.0)
```

- Ce test valide toute la chaîne de transformation : Réception JSON -> Parsing -> Renommage des clés (temp -> temperature) -> Envoi au client. Si une seule clé change, ce test échouera, nous alertant d'une régression.

Preuve du Fonctionnement du Cache

```
def test_caching_mechanism(self, mock_get):  
    # Appel 1  
    self.client.get("/get_weather?city=Lyon")  
    # Appel 2  
    self.client.get("/get_weather?city=Lyon")  
  
    # VERDICT IMPLACABLE :  
    self.assertEqual(mock_get.call_count, 1)
```

- **C'est un test de performance fonctionnelle.**
- On appelle l'URL deux fois.
- On demande au Mock : *"Combien de fois as-tu été utilisé ?"* (call_count).
- S'il répond "1", cela prouve mathématiquement que le deuxième appel a été servi par le Cache mémoire de app.py. Si c'était "2", le cache serait cassé. C'est un test extrêmement puissant pour valider l'optimisation.

Résumé Architectural

Le fichier test_suite.py est la garantie de professionnalisme du projet.

1. **Fiabilité** : Il prouve que le code gère les erreurs (404, 500, Timeout) sans crasher.
2. **Performance** : Il prouve que le cache fonctionne réellement (test call_count).
3. **Mathématiques** : Il prouve que l'algorithme adaptatif converge correctement.
4. **Maintenabilité** : Il permet à un futur développeur de modifier le code en étant sûr de ne rien casser.

C'est ce genre de fichier qui fait dire à un recruteur : *"Ce candidat ne fait pas que du*

bricolage, il construit des systèmes fiables."